



**WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA**
z siedzibą w Rzeszowie

KOLEGIUM INFORMATYKI STOSOWANEJ

Kierunek: INFORMATYKA

Dawid Gdowik
Nr albumu studenta 72413

System zarządzania asortymentem sklepu internetowego w języku Python

Prowadzący: mgr inż. Aleksander Mysakowec

Dokumentacja projektu

Rzeszów 2026

Spis treści

1	Wstęp	3
1.1	Informacje o projekcie	3
1.2	Analiza wymagań systemu	4
2	Architektura Kodu i Implementacja	5
2.1	Klasy i mechanizm hermetyzacji	5
2.2	Logika biznesowa i listy składane	6
2.3	Główny punkt wejścia i pętla menu	7
3	Prezentacja działania programu	8
3.1	Uruchomienie programu i menu główne	8
3.2	Wyświetlanie całego asortymentu	8
3.3	Filtrowanie produktów po kategorii	9
3.4	Filtrowanie asortymentu po cenie maksymalnej	9
4	Podsumowanie i wnioski	10
4.1	Wnioski końcowe	10
4.2	Możliwości rozwoju programu	10
4.3	Repozytorium GitHub i wersjonowanie	11
	Bibliografia	12
	Spis rysunków	13

Rozdział 1

Wstęp

1.1 Informacje o projekcie

Temat projektu

Tematem niniejszego projektu jest projekt oraz implementacja systemu zarządzania asortymentem wirtualnego sklepu internetowego. Aplikacja została napisana w języku Python z naciskiem na wykorzystanie paradygmatu programowania zorientowanego obiektowo (OOP), co pozwala na elastyczną i skalowalną obsługę logiki biznesowej sklepu.

Założenia projektu

Głównym założeniem jest stworzenie spójnego środowiska konsolowego, demonstrującego zaawansowane techniki programistyczne w języku Python. Projekt zakłada:

- Strukturę opartą na klasach i obiektach, uwzględniającą hierarchię dziedziczenia i polimorfizm (klasy pochodne dziedziczące po klasie bazowej).
- Hermetyzację danych przy użyciu właściwości (Gettery i Settery) w celu bezpiecznego zarządzania atrybutami (np. walidacja cen).
- Integrację natywnych struktur danych oraz optymalizację ich przetwarzania z wykorzystaniem list składanych (*list comprehensions*).
- Przejrzysty podział logiki programu z wykorzystaniem funkcji.

Zastosowanie projektu

Opracowany system znajduje zastosowanie jako uproszczony model zaplecza (back-office) dla administratora sklepu internetowego. Pozwala na:

- Zarządzanie katalogiem produktów z zachowaniem podziału na specyficzne kategorie asortymentowe (np. elektronika, odzież).
- Szybkie filtrowanie i przeszukiwanie dostępnych produktów na podstawie zadanych kryteriów.
- Demonstrację solidnej architektury kodu, która w przyszłości może stanowić fundament do rozbudowy o graficzny interfejs użytkownika (GUI) lub interfejs webowy.

1.2 Analiza wymagań systemu

Wymagania funkcjonalne

- Tworzenie i modyfikacja obiektów reprezentujących unikalne produkty w ofercie sklepu.
- Ewidencjonowanie specyficznych cech produktów w zależności od ich typu (np. deklaracja rozmiaru dla ubrań, okresu gwarancji dla sprzętu elektronicznego).
- Filtrowanie bazy asortymentu w celu wyodrębnienia produktów spełniających określone warunki (np. poniżej maksymalnej kwoty).
- Wyświetlanie sformatowanych raportów i szczegółowych informacji o stanie magazynowym poszczególnych pozycji.

Wymagania niefunkcjonalne

- **Integralność danych:** Wymuszenie poprawności wprowadzanych parametrów poprzez mechanizmy hermetyzacji (np. blokada możliwości przypisania produktom wartości ujemnych).
- **Skalowalność i polimorfizm:** Architektura kodu bazująca na klasach pozwala na łatwe dodawanie kolejnych kategorii produktów w przyszłości bez ingerencji w główny rdzeń systemu.
- **Wydajność operacji:** Zastosowanie list składanych skraca czas i złożoność pamięciową przeszukiwania list produktów względem klasycznych pętli.

Rozdział 2

Architektura Kodu i Implementacja

W niniejszym rozdziale przedstawiono strukturę implementacji systemu oraz omówiono kluczowe fragmenty kodu źródłowego odpowiedzialne za realizację założeń projektowych.

2.1 Klasy i mechanizm hermetyzacji

Na poniższym zrzucie ekranu (Rys. 2.1) przedstawiono implementację struktury klas zorientowanej obiektowo. Klasa bazowa `Produkt` definiuje uniwersalne właściwości obiektów, natomiast za pomocą dekoratorów `@property` oraz `@cena.setter` zrealizowano hermetyzację i automatyczną walidację poprawności ceny. Klasy pochodne `Elektronika` oraz `Odzież` dziedziczą po klasie bazowej i nadpisują metodę `pokaz_szczegoly()`, implementując paradygmat polimorfizmu.

```
1  class Produkt:
2      def __init__(self, nazwa, cena):
3          self.nazwa = nazwa
4          self.cena = cena
5
6      @property
7      def cena(self):
8          return self._cena
9
10     @cena.setter
11     def cena(self, wartosc):
12         if wartosc < 0:
13             print(f"[Błąd] Cena produktu '{self.nazwa}' nie może być ujemna! Ustawiono 0.0 zł.")
14             self._cena = 0.0
15         else:
16             self._cena = float(wartosc)
17
18     def pokaz_szczegoly(self):
19         return f"Produkt: {self.nazwa} | Cena: {self.cena:.2f} zł"
20
21
22     class Elektronika(Produkt):
23         def __init__(self, nazwa, cena, gwarancja_miesiace):
24             super().__init__(nazwa, cena)
25             self.gwarancja = gwarancja_miesiace
26
27         def pokaz_szczegoly(self):
28             return f"[Elektronika] {self.nazwa} | Cena: {self.cena:.2f} zł | Gwarancja: {self.gwarancja} mies."
29
30
31     class Odzież(Produkt):
32         def __init__(self, nazwa, cena, rozmiar):
33             super().__init__(nazwa, cena)
34             self.rozmiar = rozmiar
35
36         def pokaz_szczegoly(self):
37             return f"[Odzież] {self.nazwa} | Cena: {self.cena:.2f} zł | Rozmiar: {self.rozmiar}"
```

Rysunek 2.1: Implementacja klas bazowych, pochodnych oraz walidacji ceny.

2.2 Logika biznesowa i listy składane

Klasa `Sklep` odpowiada za logikę biznesową systemu oraz zarządzanie kolekcją obiektów przechowywanych w dynamicznej liście `asortyment` (Rys. 2.2). W metodach filtrujących zastosowano technologię list składanych (*list comprehensions*), która pozwala na wydajne i zwarte przeszukiwanie struktur danych w języku Python bez konieczności stosowania wielolinijkowych tradycyjnych pętli.

```
40 class Sklep:
41     def __init__(self, nazwa):
42         self.nazwa = nazwa
43         self.asortyment = []
44
45     # Dodawanie obiektu do listy
46     def dodaj_produkt(self, produkt):
47         self.asortyment.append(produkt)
48
49     # Wyświetlanie wszystkich produktów
50     def pokaz_asortyment(self):
51         print(f"\n--- Asortyment sklepu: {self.nazwa} ---")
52         for produkt in self.asortyment:
53             print(produkt.pokaz_szczegoly())
54
55     # Filtrowanie po cenie (lista składana)
56     def filtruj_tanie_produkty(self, max_cena):
57         print(f"\n--- Produkty w cenie do {max_cena} zł ---")
58         tanie = [p for p in self.asortyment if p.cena <= max_cena]
59         for p in tanie:
60             print(p.pokaz_szczegoly())
61
62     # Filtrowanie po klasie (lista składana)
63     def filtruj_tylko_odziez(self):
64         print("\n--- Tylko odzież ---")
65         ubrania = [p for p in self.asortyment if isinstance(p, Odziez)]
66         for u in ubrania:
67             print(u.pokaz_szczegoly())
68
```

Rysunek 2.2: Klasa zarządzająca asortymentem z użyciem list składanych.

2.3 Główny punkt wejścia i pętla menu

Ostatni fragment kodu źródłowego (Rys. 2.3) odpowiada za uruchomienie aplikacji oraz obsługę interakcji z użytkownikiem w czasie rzeczywistym. W bloku `if __name__ == "__main__"` następuje inicjalizacja obiektu klasy `Sklep` i zasilenie go startowymi danymi testowymi (w tym produktem o ujemnej cenie w celu przetestowania walidacji). Sterowanie aplikacją zorganizowano przy użyciu nieskończonej pętli `while True` oraz instrukcji warunkowych `if-elif-else`, które pobierają wybór użytkownika przez funkcję `input()` i przekierowują żądania do odpowiednich metod klasy sterującej.

```
70     if __name__ == "__main__":
71         moj_sklep = Sklep("SuperMarket24")
72
73         # PRODUKT DO WYWOŁANIA BŁĘDU DO SCREENA:
74         moj_sklep.dodaj_produkt(Elektronika("Zepsuty telefon", -50.0, 12))
75
76         # Reszta danych startowych
77         moj_sklep.dodaj_produkt(Elektronika("Telefon Huawei", 999.99, 24))
78         moj_sklep.dodaj_produkt(Odziez("Koszulka NIKE", 120.00, "M"))
79         moj_sklep.dodaj_produkt(Elektronika("Słuchawki bezprzewodowe", 149.99, 12))
80
81         # Główne menu programu
82         while True:
83             print("\n=== MENU SKLEPU ===")
84             print("1. Pokaż cały asortyment")
85             print("2. Pokaż tylko odzież")
86             print("3. Pokaż produkty do danej kwoty")
87             print("4. Zakończ program")
88
89             wybor = input("Wybierz opcję (1-4): ")
90
91             if wybor == '1':
92                 moj_sklep.pokaz_asortyment()
93             elif wybor == '2':
94                 moj_sklep.filtruj_tylko_odziez()
95             elif wybor == '3':
96                 kwota = float(input("Podaj maksymalną cenę: "))
97                 moj_sklep.filtruj_tanie_produkty(kwota)
98             elif wybor == '4':
99                 print("Koniec programu.")
100                 break
101             else:
102                 print("Niepoprawny wybór, spróbuj ponownie.")
```

Rysunek 2.3: Główny punkt wejścia do programu oraz implementacja pętli menu.

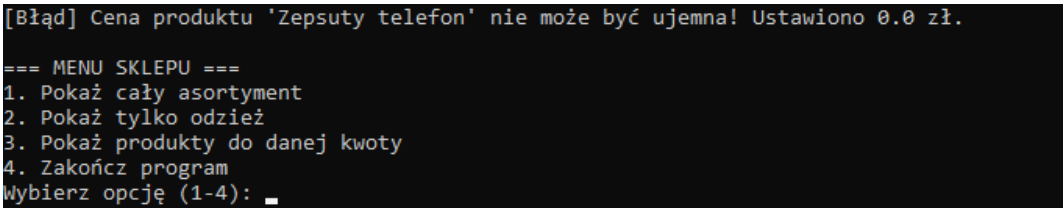
Rozdział 3

Prezentacja działania programu

W tym rozdziale pokazano, jak aplikacja działa w konsoli i jak użytkownik może korzystać z przygotowanego menu. Wszystkie opcje zostały przetestowane na przygotowanych wcześniej danych.

3.1 Uruchomienie programu i menu główne

Po odpaleniu programu w konsoli najpierw uruchamia się automatyczna walidacja ceny. Do kodu celowo dopisano testowy produkt „Zepsuty telefon” z ceną –50 zł. Jak widać na Rys. 3.1, program od razu wykrył błąd, zmienił ujemną cenę na bezpieczne 0.00 zł i wyświetlił komunikat ostrzegawczy. Zaraz pod nim pojawia się menu główne z czterema opcjami do wyboru.



```
[Błąd] Cena produktu 'Zepsuty telefon' nie może być ujemna! Ustawiono 0.0 zł.  
=== MENU SKLEPU ===  
1. Pokaż cały asortyment  
2. Pokaż tylko odzież  
3. Pokaż produkty do danej kwoty  
4. Zakończ program  
Wybierz opcję (1-4): _
```

Rysunek 3.1: Wyświetlenie komunikatu o błędzie ceny oraz menu głównego programu.

3.2 Wyświetlanie całego asortymentu

Po wpisaniu cyfry 1 i wciśnięciu Enter program pokazuje na ekranie pełną listę produktów znajdujących się w sklepie (Rys. 3.2). Na wydruku widać podział na [Elektronika] oraz [Odzież]. Każdy typ produktu ma swoje własne oznaczenia – elektronika pokazuje czas gwarancji w miesiącach, a odzież rozmiar (np. koszulka NIKE ma rozmiar M). Widać też, że cena zepsutego telefonu została na stałe ustawiona jako 0.00 zł.


```

=== MENU SKLEPU ===
1. Pokaż cały asortyment
2. Pokaż tylko odzież
3. Pokaż produkty do danej kwoty
4. Zakończ program
Wybierz opcję (1-4): 1

--- Asortyment sklepu: SuperMarket24 ---
[Elektronika] Zepsuty telefon | Cena: 0.00 zł | Gwarancja: 12 mies.
[Elektronika] Telefon Huawei | Cena: 999.99 zł | Gwarancja: 24 mies.
[Odzież] Koszulka NIKE | Cena: 120.00 zł | Rozmiar: M
[Elektronika] Słuchawki bezprzewodowe | Cena: 149.99 zł | Gwarancja: 12 mies.

```

Rysunek 3.2: Widok konsoli po wybraniu opcji pokazania całego asortymentu.

3.3 Filtrowanie produktów po kategorii

Wpisanie w menu cyfry 2 pozwala odfiltrować z całej listy wyłącznie produkty będące odzieżą. Jak widać na Rys. 3.3, po wybraniu tej opcji program pominął telewizor i słuchawki, a na ekranie wypisał tylko koszulkę NIKE wraz z jej rozmiarem i ceną.

```

=== MENU SKLEPU ===
1. Pokaż cały asortyment
2. Pokaż tylko odzież
3. Pokaż produkty do danej kwoty
4. Zakończ program
Wybierz opcję (1-4): 2

--- Tylko odzież ---
[Odzież] Koszulka NIKE | Cena: 120.00 zł | Rozmiar: M

```

Rysunek 3.3: Wynik działania filtra wyświetlającego tylko odzież.

3.4 Filtrowanie asortymentu po cenie maksymalnej

Ostatnia funkcja testowa to opcja numer 3, czyli szukanie produktów do określonej kwoty. Na Rys. 3.4 pokazano sytuację, w której wpisano maksymalną cenę równą 120 zł. Program automatycznie przeszukał listę i wyświetlił tylko te rzeczy, które mieszczą się w tym limicie: telefon (za 0 zł) oraz koszulkę NIKE (równie 120 zł). Droższe przedmioty zostały prawidłowo schowane.

```

=== MENU SKLEPU ===
1. Pokaż cały asortyment
2. Pokaż tylko odzież
3. Pokaż produkty do danej kwoty
4. Zakończ program
Wybierz opcję (1-4): 3
Podaj maksymalną cenę: 120

--- Produkty w cenie do 120.0 zł ---
[Elektronika] Zepsuty telefon | Cena: 0.00 zł | Gwarancja: 12 mies.
[Odzież] Koszulka NIKE | Cena: 120.00 zł | Rozmiar: M

```

Rysunek 3.4: Lista produktów odfiltrowana według wpisanej ceny maksymalnej.

Rozdział 4

Podsumowanie i wnioski

W ramach projektu udało się stworzyć działający program do zarządzania produktami w sklepie internetowym. Aplikacja została napisana w języku Python z wykorzystaniem programowania obiektowego i pozwala użytkownikowi na wygodną obsługę przez menu w konsoli.

4.1 Wnioski końcowe

Podczas pisania i testowania programu udało się sformułować następujące wnioski:

- **Wykorzystanie obiektowości:** Dzięki zastosowaniu dziedziczenia i polimorfizmu cały kod jest dobrze poukładany. Program potrafi bez problemu zarządzać różnymi typami produktów (elektronika i odzież) w ramach jednej wspólnej listy.
- **Bezpieczeństwo i walidacja:** Hermetyzacja, czyli użycie getterów i setterów, pozwoliła skutecznie zabezpieczyć program przed błędami. System sam pilnuje, żeby nikt nie wpisał ujemnej ceny i automatycznie poprawia ją na zero.
- **Skrócenie kodu:** Użycie list składanych (*list comprehensions*) do filtrowania asortymentu sprawiło, że kod jest o wiele krótszy, bardziej czytelny i działa szybciej niż tradycyjne, długie pętle.

4.2 Możliwości rozwoju programu

Przygotowana struktura aplikacji pozwala na jej łatwą rozbudowę w przyszłości. Program można w prosty sposób rozwinąć o dodatkowe funkcjonalności, takie jak:

- Zapisywanie i wczytywanie produktów z plików zewnętrznych, na przykład w formacie JSON lub tekstowym.
- Stworzenie okienkowego interfejsu użytkownika (GUI) zamiast czarnej konsoli, na przykład przy użyciu wbudowanej biblioteki Tkinter.

Podsumowując, aplikacja spełnia wszystkie założenia projektowe, działa stabilnie i poprawnie reaguje na podstawowe błędy użytkownika. Stanowi ona dobrą bazę pod większy system e-commerce.

4.3 Repozytorium GitHub i wersjonowanie

Podczas pracy nad projektem korzystałem z systemu kontroli wersji Git. Wszystkie pliki źródłowe programu, wraz z pełną historią wprowadzanych zmian, zostały wrzucone do zdalnego repozytorium w serwisie GitHub. Kod aplikacji jest dostępny publicznie pod adresem: <https://github.com/dawidgdowik1/ProjektPython2026>. Dzięki temu projekt jest bezpiecznie zarchiwizowany i łatwy do podejrzenia przez przeglądarkę.

Bibliografia

- [1] Lutz M., *Python. Wprowadzenie*, Wydawnictwo Helion, Gliwice.
- [2] Ramalho L., *Zaawansowany Python*, Wydawnictwo O'Reilly / Helion, Gliwice.
- [3] Python Software Foundation, *Python Language Documentation*, <https://docs.python.org/>, [Dostęp: maj 2026].
- [4] Martin R. C., *Czysty kod. Podręcznik dobrego programisty*, Wydawnictwo Helion, Gliwice.
- [5] *Materiały dydaktyczne do zajęć z języka Python*, platforma Moodle WSliZ, https://moodle.wsiz.edu.pl/pluginfile.php/502800/mod_resource/content/1/v2_Python.pdf, [Dostęp: maj 2026].
- [6] Mysakowec A., *Materiały do zajęć laboratoryjnych z przedmiotu Programowanie w języku R i Python*, Kolegium Informatyki Stosowanej, Wyższa Szkoła Informatyki i Zarządzania w Rzeszowie, rok akademicki 2025/2026.

Spis rysunków

2.1	Implementacja klas bazowych, pochodnych oraz walidacji ceny.	5
2.2	Klasa zarządzająca asortymentem z użyciem list składanych.	6
2.3	Główny punkt wejścia do programu oraz implementacja pętli menu.	7
3.1	Wyświetlenie komunikatu o błędzie ceny oraz menu głównego programu.	8
3.2	Widok konsoli po wybraniu opcji pokazania całego asortymentu.	9
3.3	Wynik działania filtra wyświetlającego tylko odzież.	9
3.4	Lista produktów odfiltrowana według wpisanej ceny maksymalnej.	9